

设计报告

小车跟随行驶系统（C 题）

摘要

智能和科技发展已经成为当今社会的重要发展方向，生活中的智能小车无处不在，并由此延伸到更宽的领域，例如军事和科研领域，本次设计的小车跟随行驶系统，由领头小车和跟随小车共同组成，采用的是 TI 公司 M430F5438A 型号为主控制器驱动简易装置，以及寻迹模块装置，该装置包括 Open MV 岔路口识别和 Open MV 寻迹模块，电源模块，电机驱动模块，除此之外还加舵机转向，OLED 显示，蜂鸣器报警，红外辅助检测停止符模块。

关键词：M430F5438A，Open MV 模块，小车跟随系统

Abstract

Intelligent and technological development has become an important development direction of today's society, life in the intelligent car everywhere, and thus extended to a wider range of fields, such as military and scientific research, the design of the car to follow the driving system, by the lead car and follow the car together, the use of TI M430F5438A model is the main controller drive simple device, as well as the tracing module device, the The device includes Open MV fork recognition and Open MV tracing module, power supply module, motor drive module, in addition to refuelling rudder steering, OLED display, buzzer alarm, infrared assisted detection stop sign module.

Keywords: M430F5438A, intelligent car, Open MV

一、方案论证与比较

1.1 任务要求及解读

本题要求设计相应电路装配小车，并且利用 TI 的 MCU 作为主控。领头小车实现在指定的地图上以 $0.3 \sim 1\text{m/s}$ 的速度进行寻迹和跟随小车协同作业，领头小车与跟随小车的通信方式，距离控制，岔路口和单普通情况下的识别，以及最基础的寻迹功能，本题需要我们对相关的传感器，以及 openMV 系统熟悉使用。

1.2 系统方案

本系统主要对 Open MV 寻迹模块，红外识别停止位和电机驱动芯片，对这两个模块进行选择，使其电路更运行更加安全和有效，完成题目指定操作。

1.3 寻迹模块论证与选择

方案一：TCRT5000 红外寻迹模块

本产品采用 TCRT5000 红外反射传感器，一种集发射与接收于一体的光电传感器，它由一个红外发光二极管和一个 NPN 红外光电三极管组成。可以检测白底中的黑线，也可以检测黑底中的白线，利用红外线光遇到白色物体表面具有不同的反射性质的特点，在小车行驶过程中不断地向地面发射红外光，当红外光遇到白色纸质地板时发生漫反射，反射光被装在小车上的接收管接收，如果遇到黑线，则红外光被吸收小车上的接收管接收不到红外光，单片机就是是否收到反射回来的红外光为依据来确定黑线的位置和小车的行走路线，从而实现小车的寻迹功能可用作寻线小车的必备传感器。但是检测反射距离 $1\text{mm} \sim 25\text{mm}$ 适用，检测距离要求较近

方案二：Open MV

Open MV 是一个开源，低成本，功能强大的机器视觉模块。以 STM32F427CPU 为核心，集成了 OV7725 摄像头芯片，在小巧的硬件模块上，用 C 语言高效地实现了核心机器视觉算法，提供 Python 编程接口，可以用 Python 语言使用 Open MV 提供的机器视觉功能，所带例程有现成巡线程序并回传截距和角度误差，十分方便。

综上所述，红外寻迹模块对本题来说，识别精度有限，对光线以及比赛地图的颜色材料要求较高，需要反复调整滑动电阻，以提高识别精度，使用过程相对复杂；Open MV 及其方便快捷，故最终选择 Open MV 模块作为小车的寻迹模块。

1.5 停止标识符号的识别模块论证与选择

方案一：Open MV

我们本想采用 OpenMV 同时实现巡线和停止标识符的识别，将 OpenMV 的窗口分块转换成黑白二值图像进行识别，当中间的狭长的小窗口识别到黑色区域（目标颜色为白色）时向主控回传数据，但识别停止标识符时很容易与识别赛道相互误判，再去利用其他算法较为复杂

方案二：TCRT5000 红外寻迹模块

采用 TCRT5000 红外反射传感器，一种集发射与接收于一体的光电传感器，它由一个红外发光二极管和一个 NPN 红外光电三极管组成。可以检测白底中的黑线，也可以检测黑底中的白线，利用红外线光遇到白色物体表面具有不同的反射性质的原理来识别黑色会白色

综上所述，在深入研究两个模板后，发现 Open MV 修改算法识别停止标识符较为复杂，而利用红外寻迹模块程序方面相对简单，所以选择 TCRT5000

1.6 电机驱动芯片论证与选择

方案一：L298N 驱动板

L298N 就是 L298 得立式封装，是一款可接受高电压、大电流双路全桥式电机驱动芯片，工作电压可达 46V，输出电流最高可至 4A，采用 Multiwatt15 脚封装，接受标准 TTL 逻辑电平信号，具有两个使能控制端，在不受输入信号影响的情况下通过板载跳帽插拔的方式，动态调整电路运作方式，有一个逻辑电源输入端，通过内置的稳压芯片 78M05，使内部逻辑电路部分在低电压下工作。

方案二：BTN7971B 驱动板

BTN7971B 是用于电机驱动应用的集成大电流半桥。它是一个 P 沟道高侧 MOSFET 和一个 N 沟道低侧 MOSFET，其中一个封装有集成驱动芯片。由于 p 通道高压侧开关，因此无需电荷泵，从而将电磁干扰降至最低。集成驱动芯片易于与微控制器接口，该芯片具有逻辑电平输入、电流检测诊断、转换率调整、死区时

间生成和过热、过压、欠压、过流和短路保护功能。BTN7971B 提供了一个成本优化的解决方案,用于具有非常低的板空间消耗的受保护的大电流 PWM 电机驱动器。

综上所述,基于对 L298N 驱动板散热量低,发热量低,抗干扰能力强,驱动能力强,有较强的可靠度,小组同学选择了 L298N 作为驱动模块。

二、理论分析与计算

2.1 Open MV 寻迹模块的实现

结合题目要求，由于 Open MV 是一个开源，低成本，功能强大的机器视觉模块。以 STM32F427CPU 为核心，集成了 OV7725 摄像头芯片，在小巧的硬件模块上，用 C 语言高效地实现了核心机器视觉算法，提供 Python 编程接口，可以用 Python 语言使用 Open MV 提供的机器视觉功能。数字摄像头将采集的图像传给 Open MV 模块进行图像信息的处理，最高图像传输为 60 帧每秒，足够保证图像采集速率。

图像采集或传输过程中受到很多因素的影响，会产生很多噪声，所以在对图像进行识别与检测时，要对采集的图像进行预处理操作。图像的预处理通常包括图像的灰度化，对图像滤波进行降噪处理，对图像进行二值化处理，在以此为前提之下，即可实现 Open MV 的单线路寻迹识别，对于只有单条黑色导引线的识别与追踪，采用直线检测的方法。由 Open MV 采集的图像经图像滤波后，对图像进行色块检测和 Otsu 算法二值化处理，寻找到黑色引导线，随后对引导线进行线性回归处理，获得该线段的直线参数，判断是否检测到直线，如果不是，则放弃该帧图像，返回上一步，取下一帧图像；如果是，计算出该直线与小车的偏角，转换成小车应该转角的角度。将数值通过通信接口传输给 ESP32 控制器，从而控制舵机转角，即可完成寻迹。

2.2 通讯分析与设计

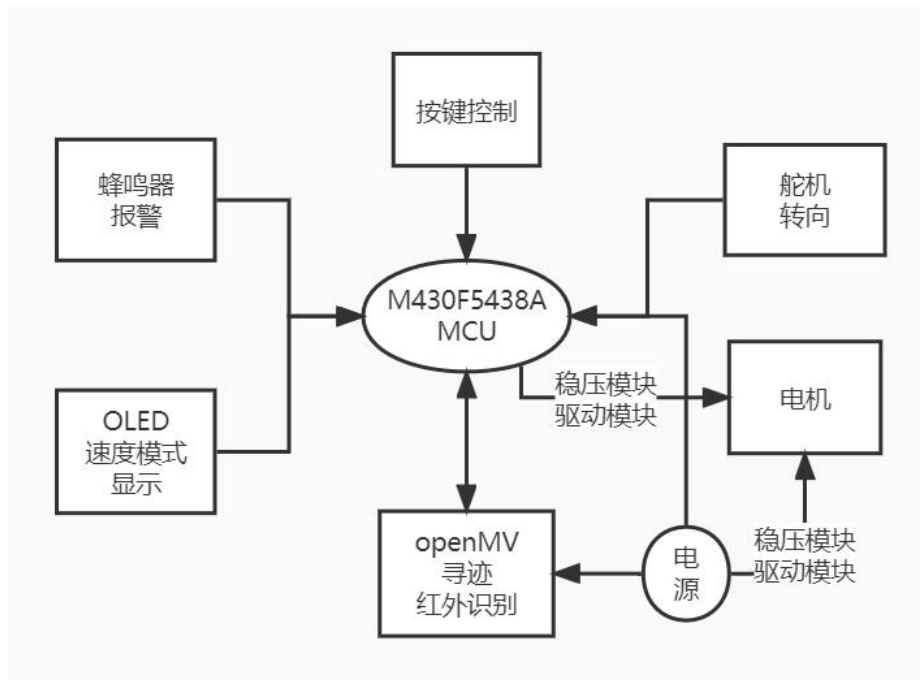
利用 NRF24L01P 2.4G 射频收发模块实现领头小车和跟随小车的之间的相互通讯，在领头小车和跟随小车上分别连上 2.4G 收发模块，实现两车实时的数据交换

三、电路设计与程序设计

3.1 系统组成

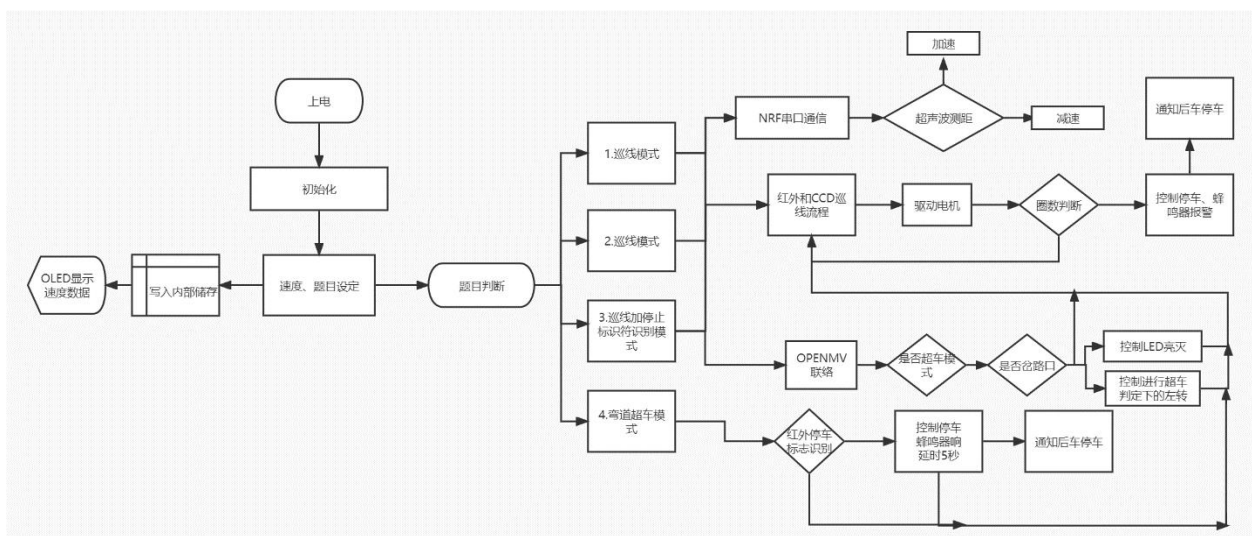
要系统由 M430F5438A 芯片、Open MV 图像处理和红外辅助识别，舵机转向模块，OLED 显示，蜂鸣器报警，按键控制，以及电机驱动板和稳压板这组成，其余的就是电源对 OpenMV, 电机，主控的单独供电

3.2 原理功能图



3.3 系统软件与程序流程

如下两图分别是整个系统的基本基本流程图，由流程图的逻辑在 keil 上用 C 语言进行编译，再通过串口下载



四、测试方案与测试结果

4.1 测试方案

测试一：

我们根据题目一，将领头小车放在起始点 A，跟随小车方程 20cm 后，让领头小车速度为 0.3m/s, 让两辆小车行驶外圈一圈停止。

要求：

(1) 领头小车平均速度小于 10%，(2) 两车间距 20cm 且不能发生碰撞，(3) 当行驶一圈到达 A 后，停止时间不能超过 1s, 跟随小车与领头小车距离在 14~26cm 范围内

测试二：

按照题目二要求，将领头小车放在路径轨迹的起始位置 A 点，跟随小车放在路径上 E 点所在边的直线区域，由测试专家指定的位置，设定领头小车速度为 0.5m/s，沿着外圈路径行驶两圈停止。

要求：

(1) 领头小车平均速度小于 10%，(2) 运行过程中两车间距 20cm 且不能发生碰撞(3) 当行驶一圈到达 A 后，停止时间不能超过 1s, 跟随小车与领头小车距离在 14~26cm 范围内

测试三：

按照题目三要求，在路径的 E 点所在边的直线区域任意位置，放上“等停指示”标识。然后，将领头小车放在路径的起始位置 A 点，跟随小车放在其后 20cm 处，设定领头小车速度为 1m/s，沿着外圈路径行驶一圈，行驶中两小车不得发生碰撞。

要求：

(1) 领头小车平均速度小于 10%(2) 在停止标识符停车且误差不大于 5cm, 同时跟随小车停止，时间不超过 1s，车距在 14~26cm 范围内(3) 领头小车在“等停指示”停止 5s 后继续行驶到 A 点，跟随小车跟随行驶到停车点停车停止时间不能超过 1s, 跟随小车与领头小车距离在 14~26cm 范围内

测试四：

按题目四，将领头小车放在路径的起始位置 A 点，跟随小车放在其后 20cm 处，领头小车和跟随小车连续完成三圈路径的行驶。第一圈领头小车和跟随小车都沿着外圈路径行驶。第二圈领头小车沿着外圈路径行驶，跟随小车沿着内圈路径行驶，实现超车领跑。第三圈跟随小车沿着外圈路径行驶，领头小车沿着内圈路径行驶，实现反超和再次领跑。

要求：

(1)两车行驶平稳且不发生碰撞(2) 完成三圈形式后，领头小车到达一点停止跟随小车及时停止，两车停止，时间差不超过一秒，车距在 14~26cm 范围内。(3) 小车行驶速度不得低于 0.3m/s，且完成所规定的三圈轨迹行驶所需时间越短越好。

4.2 测试结果分析

上述共四个部分的测试，测试表格如下

	题目一	题目二	题目三	题目四
设定速度	0.3m/s	0.5m/s	>0.3m/s	1m/s
任务用时	19s	24.1s	41.5s	7.3s
总路径长度	5.48m	10.96m	15.2m	4.8m
领头小车平均速度	0.34m/s	0.29m/s	0.38m/s	0.3m/s
两车间距	5.5cm	11cm	15.8cm	5.5cm
停止时间差	0.7s	0.9s	0.6s	0.8s
A 点是否停车	是	是	是	是
指示位是否停车 5s 后行驶	\	\	\	是
A 点蜂鸣器是否响起	是	是	是	是
BFD 灯是否打	\	\	是	\

开				
是否符合要求	是	是	是	否

以上为小车的测试数据。

五、感悟与参赛总结

首先感谢重庆市大学生电子设计大赛组委会和各协办单位给我们提供了这一次宝贵的锻炼机会，在这短暂的一个月时间里，我们的团队齐心协力，在指导老师的指引下学到了许多。与同学们的交流让我明白了团队协作的重要性，并发现自身的不足。这一个月里早起晚归，生活在实验室让我感受到了课堂之外，实验室之中的快乐。在这比赛的四天三夜里，我们的团队分工明确，迎难而上，互相鼓励，共担难题，四天三夜经历了调试 BUG 的痛苦，突破难题的喜悦，这让我们相信未来的道路总是风风雨雨，学习的过程总是充满磨难，只要用心体会，总会有精彩绽放的瞬间，只要积极参与其中，一定会有所收获！

六、参考文献

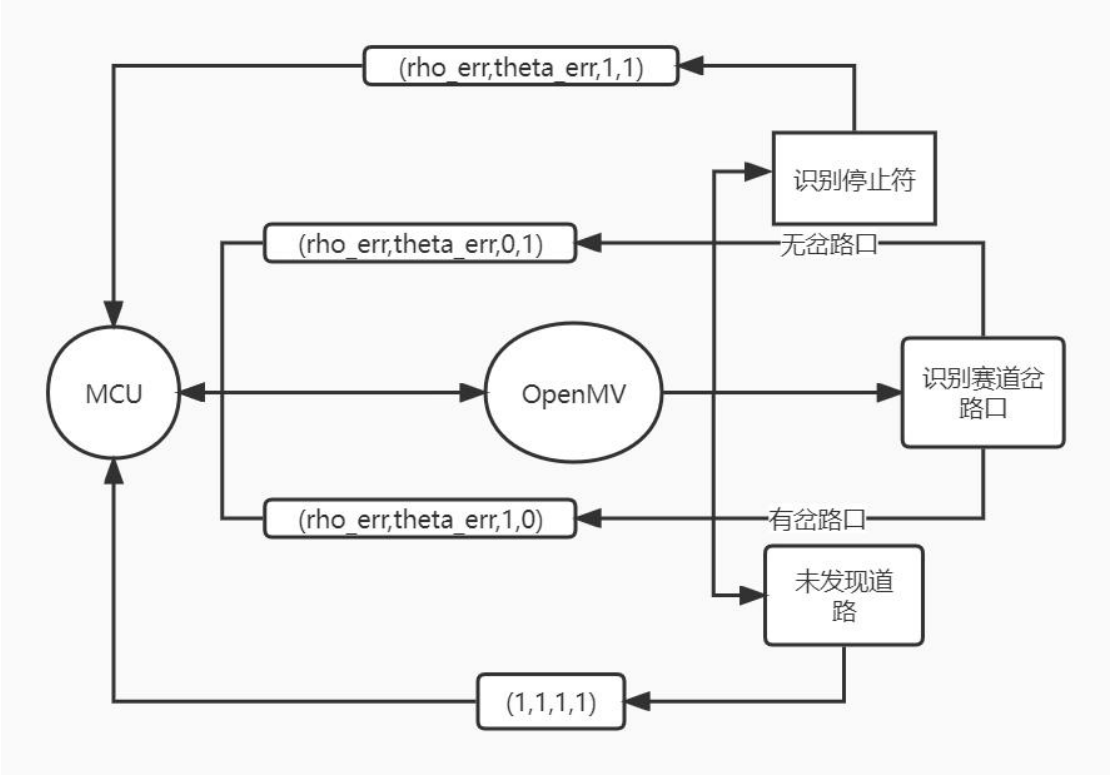
- [1] 蔚利娜. 基于加速度的计步算法和步长计算研究与实现[D]. 沈阳: 东北大学, 2013:6-1.
- [2] 白冰雪, 张延林, 单片机原理及应用[M]. 哈尔滨: 东北林业大学出版社, 2006.
- [1] 蔚利娜. 基于加速度的计步算法和步长计算研究与实现[D]. 沈阳: 东北大学, 2013:6-1.
- [2] 白冰雪, 张延林, 单片机原理及应用[M]. 哈尔滨: 东北林业大学出版 2006.
- [3] 朱嵘涛. 基于增量式 PID 算法的直流电机调速系统. 长江大学工程技术学院, 湖北荆州, 434020
- [4] 沈中坤. 基于 OpenMV 视觉模块和 MPU6050 角度传感器的智慧寻路小车. 南京信息工程大学电子与信息工程学院, 江苏南京, 210000
- [5] 解梦达. 场景语义引导下的改进灰度世界算法. 中国刑事警察学院公安信息技术与情报学院, 沈阳, 110854

七、附录

附录一：元器件明细表

1. M430F5438A 开发板
2. OpenMV4 H7 Pluse 寻迹模块
3. AMS1117-3.3V 5V 降压稳压模块
4. L298N 电机驱动模块
5. N20 直流减速电机
6. SG90 舵机
7. 有源蜂鸣器
8. 按键
9. 杜邦线

附录二：OpenMV 识别流程图



附录三：源代码

Openmv:

```
THRESHOLD = (4, 25, -20, 6, -2, 26)

import sensor, image, time, ustruct
from pyb import UART, LED
import machine
import pyb

LED(1).on()
LED(2).on()
LED(3).on()

sensor.reset()
sensor.set_vflip(True)
sensor.set_hmirror(True)
sensor.set_pixformat(sensor.RGB565)
sensor.set_framesize(sensor.QQVGA) # 80x60(4, 800 pixels) - 0(N ^ 2) max = 2, 3040, 000.
sensor.skip_frames(time = 2000) # WARNING: If you use QQVGA it may take seconds
clock = time.clock() # to process a frame sometimes.

uart = UART(3, 115200) # 定义串口3变量
uart.init(115200, bits = 8, parity = None, stop = 1) # init with given parameters

# 识别区域
roil = [(0, 1, 20, 22), # 左上 x y w h
(0, 33, 20, 22), # 左下
(30, 0, 22, 18), # 上
(0, 0, 80, 60),
(56, 33, 25, 22)] # 停车

def outuart(x, a, flag) : # 用于给主控传递参数
global uart;
f_x = 0 #这两个值用于判断是单线(01)还是双线(10)
f_a = 0 #
if flag == 0:
pass
elif flag == 1 : #拐弯或直线(单线)
x, a, f_x, f_a = (rho_err, theta_err, 0, 1)
elif flag == 2: # 上左
x, a, f_x, f_a = (rho_err, theta_err, 1, 0) #有弯道时
elif flag == 3: #直走
x, a, f_x, f_a = (x, a, 1, 1)
```



```

elif flag == 4: # stop
x, a, f_x, f_a = (x, a, 1, 1)

uart.write(str("%+04d:%+04d:%d:%d" % (x, a, f_x, f_a)) # 必须要传入一个字节数组
print(str("%+04d:%+04d:%d:%d" % (x, a, f_x, f_a))

p9_flag = 0 # p9需检测从低变高
not_stop = 0

while True:
clock.tick()
img = sensor.snapshot().binary([THRESHOLD])
line = img.get_regression([(100, 100)], robust = True)

left_up_flag, left_down_flag, up_flag = (0, 0, 0)
for rec in roi1 :
img.draw_rectangle(rec, color = (255, 0, 0)) # 绘制出roi区域
#p = pyb.Pin("P9", pyb.Pin.IN)
p = machine.Pin("P9", machine.Pin.IN)

if (line) :
    rho_err = abs(line.rho()) - img.width() / 2
    if line.theta() > 90:
theta_err = line.theta() - 180
    else :
        theta_err = line.theta()
        # 直角坐标调整
        img.draw_line(line.line(), color = 127)

    outdata = [rho_err, theta_err, 0]
    print(outdata)

    if img.find_blobs([(96, 100, -13, 5, -11, 18)], roi = roi1[0]) : # 左上
        left_up_flag = 1
    else:
left_up_flag = 0
if img.find_blobs([(96, 100, -13, 5, -11, 18)], roi = roi1[1]) : # 左下
left_down_flag = 1
else :
    left_down_flag = 0
    if img.find_blobs([(96, 100, -13, 5, -11, 18)], roi = roi1[2]) : # up
        up_flag = 1
    else :
        up_flag = 0

```

```

if img.find_blobs([(96, 100, -13, 5, -11, 18)], roi = roi1[4]) : # 左下
    right_down_flag = 1
else :
    right_down_flag = 0
    if up_flag == 1 and left_up_flag == 0 and left_down_flag == 0 and right_down_flag
== 0 :

        outuart(0, 0, 3)
        if (left_up_flag and up_flag) == 0 : #只有一条线
            outuart(0, 0, 1)
            # time.sleep_ms(200)
            print('single line')
            continue
            if (left_up_flag and left_down_flag and up_flag) == 1:
outuart(0, 0, 2)
#time.sleep_ms(200)
print('two lines')
if (left_up_flag and right_down_flag) == 1:
time.sleep_ms(200)
if (left_down_flag and up_flag) == 1 :
    time.sleep_ms(250)
    outuart(0, 0, 4)
    stop_flag = 1
    print('stop')
    while stop_flag == 1 :
        if p.value() == 0 :
            stop_flag = 0
            time.sleep_ms(1000)
            pass
            #continue
        else:
pass
else:
    pass

```

主控程序 main.c

```
#include "main.h"
#include "dma.h"
#include "spi.h"
#include "tim.h"
#include "usart.h"
#include "gpio.h"
#include "control.h"
#include "pid.h"
#include <math.h>
#include <stdio.h>
#include <string.h>
#include "oled.h"
#include "delay.h"
#include "nrf24l01.h"

float servo_sum = 0;
uint8_t tim4_flag = 0;
uint8_t txbuff[32]={'1','2','3'};
int main(void)
{
    HAL_Init();
    SystemClock_Config();

    MX_GPIO_Init();
    MX_DMA_Init();
    MX_TIM2_Init();
    MX_TIM3_Init();
    MX_USART1_UART_Init();
    MX_USART3_UART_Init();
    MX_TIM4_Init();
    MX_TIM1_Init();
    MX_SPI1_Init();

    MX_NVIC_Init();

    OLED_Init();
    OLED_Clear();
    OLED_ShowString(1,1,"PWM:");
    OLED_ShowString(2,1,"Speed:");
    OLED_ShowString(2,1,"Speed:");
    OLED_ShowString(3,1,"Mode:");
```

```

OLED_ShowString(2, 7, "0.3m/s");
OLED_ShowNum(1, 5, set_pwm, 3);

OLED_ShowString(4, 2, "ok");
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_4);
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_2);
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_3);
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_4);
HAL_Delay(1000);
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_2);
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_3);
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_4);
HAL_TIM_Base_Start_IT(&htim4);

set_zhuanwan_output(0);

while (1)
{

    if(process_flag)
    {
        PROCESS_MOTOR();
        process_flag=0;
    }

    key_proce();
}

}

void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
    RCC_OscInitStruct.HSEState = RCC_HSE_ON;
    RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
    RCC_OscInitStruct.HSIState = RCC_HSI_ON;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
    RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)

```

```

{
    Error_Handler();
}
RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                               |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
{
    Error_Handler();
}
}
static void MX_NVIC_Init(void)
{
    HAL_NVIC_SetPriority(USART1_IRQn, 2, 0);
    HAL_NVIC_EnableIRQ(USART1_IRQn);
}
int fputc(int ch, FILE *f)
{
    HAL_UART_Transmit(&huart3, (uint8_t *)&ch, 1, 0xffff);
    return ch;
}
void Error_Handler(void)
{
    __disable_irq();
    while (1)
    {
    }
}

```